```python
import sys

class Customer:
    """Customer class with bank operations."""
    bank_name = 'State Bank of India'

    def __init__(self, name, balance=0.0):
        self.name = name
        self.balance = balance

    def deposit(self, amt):
        self.balance += amt
        print(f'Balance after deposit: {self.balance}')

    def withdraw(self, amt):
        if amt > self.balance:
            print('Insufficient Funds.. cannot perform this operation')
            return  # Using return instead of sys.exit() to keep the
program running
        self.balance -= amt
        print(f'Balance after withdraw: {self.balance}')

# Program Execution
print(f'Welcome to {Customer.bank_name}')
name = input('Enter Your Name: ')
c = Customer(name)

while True:
    print('\nd-Deposit \nw-Withdraw \ne-Exit')
    option = input('Choose your option: ').lower() # Simplified case
handling

    if option == 'd':
        amt = float(input('Enter amount: '))
        c.deposit(amt)
    elif option == 'w':
        amt = float(input('Enter amount: '))
        c.withdraw(amt)
    elif option == 'e':
        print('Thanks for Banking')
        sys.exit()
    else:
        print('Invalid option.. Please choose a valid option')
```

```
C:\Users\admin\AppData\Local\Programs\Python\Python313\python.exe C:\U
Welcome to State Bank of India
Enter Your Name: Somen Sridhar

d-Deposit
w-Withdraw
e-Exit
Choose your option: d
Enter amount: 100000
Balance after deposit: 100000.0

d-Deposit
w-Withdraw
e-Exit
Choose your option: w
Enter amount: 200000
Insufficient Funds.. cannot perform this operation

d-Deposit
w-Withdraw
e-Exit
Choose your option: e
Thanks for Banking

Process finished with exit code 0
```

1. The `Customer` Class

This serves as a "blueprint" for every person who opens an account.

- **Class Attribute (`bank_name`):** This is shared by all customers. Every customer belongs to the "State Bank of India."
- **Constructor (`__init__`):** When you create a customer (`c = Customer(name)`), it sets their name and starts their balance at `0.0` by default.
- **Methods (`deposit` & `withdraw`):** These define the actions a customer can take.
  - The **deposit** method adds money to the `self.balance`.
  - The **withdraw** method checks if the user has enough money before subtracting it. If they don't, it shows an "Insufficient Funds" message.

2. The Main Program Loop (`while True`)

This section handles the **User Interface (UI)** and makes the program interactive.

- **Input Gathering:** It asks for your name once at the start and creates your account object (`c`).
- **Infinite Loop:** The `while True` ensures the menu keeps appearing until you explicitly choose to exit ('e').
- **Option Handling:**
  - It uses `.lower()` on the input so that entering 'D' or 'd' both work.
  - It converts the amount input into a **float** (decimal number) because money isn't always a whole number.

3. Key Logic Points

- **State Management:** The variable `self.balance` keeps track of your money across different deposits and withdrawals.
- **Validation:** In the `withdraw` method, the `if amt > self.balance:` check prevents the account from going into a negative balance.
- **F-Strings:** Used in the `print` statements (e.g., `f"Balance: {self.balance}"`), these allow Python to "plug" the variable value directly into the text for clean reporting.

4. Exit Strategy

- **`sys.exit()`:** This is a clean way to shut down the Python script entirely when the user chooses the 'Exit' option.